Presentation Guide and Panel Q&A

Uplift: Student Success Assistant · 10-minute panel presentation

For the presenter. Slide-by-slide talk track, live demo script, key numbers, glossary and answers to likely questions. Numbers are read from the evaluation artifacts when this guide is built, so they match the deck and reports.

1. Timing plan (10 minutes)

#	Slide	Time	Goal
1	Title	0:15	Name, one-line pitch
2	Problem statement	0:40	The advisor's problem and the three questions in the brief
3	Business need	0:35	Why early, explainable and capacity-aware matters
4	Solution overview	0:40	What Uplift does, in one picture
5	Architecture	1:00	Offline build, online request, agents, RAG, LLM tiers
6	Data flow	0:40	From OULAD to a support plan; the leakage rule
7	Technology stack	0:35	Each choice with its reason
8	Agent workflow	0:45	Roles, critic, revision, fallback
9	Hybrid RAG	0:40	Three channels, filters, results vs random
10	Prediction models	0:40	Honest metrics and why
11	Demo	2:00	Live: week clock, student, plan, case finder, copilot
12	Evaluation results	0:50	DeepEval, RAGAS, judge, tests
13	Responsible AI	0:30	Guardrails, sensitive attributes, fairness
14	Challenges and learnings	0:30	What was hard and what changed
15	Thank you / Q&A	0:10	Invite questions

2. Talk track

1. Title. "Uplift is an AI student-success assistant for Open University module teams. It finds students who may withdraw or fail while there is still time, shows what happened to similar students before, and drafts a supportive plan that an advisor reviews."

2. Problem statement. Advisors have thousands of records and little time. The brief asks three questions: have similar students struggled before, which factors are behind poor outcomes, and who should be supported first. Dashboards answer "who" but not "why", "what happened before" or "what to do".

3. Business need. Contacting students early is what changes outcomes, but capacity is limited: if an advisor can call 10% of a cohort, the list must be right at the top. Advisors also need reasons they can defend and messages that are supportive, never punitive. At week 15, 11.3% of students still enrolled will

go on to withdraw; contacting the top 10% by our ranking reaches students who fail or withdraw 98% of the time.

4. Solution overview. Four things in one console: early warning with reasons, a ranked contact list, similar historical students (hybrid RAG), and a multi-agent support plan with an outreach draft. Plus a copilot, insights and evaluation. Everything is "as of" a week in the course: weeks 5, 9, 15, 22.

5. Architecture. Bottom: the offline build: dataset, preprocessing, one document per enrolment, embeddings, LanceDB plus BM25, and model training, once per checkpoint. Middle: the request path: React console, FastAPI with input guardrails, then the support-plan workflow: evidence assembled in code from the models and the retriever; the LangGraph agents (performance, risk, router and intervention, critic with one revision); then the precedent synthesis. Claude Sonnet drafts text; code decides facts. Every LLM step has a computed fallback. Evaluation and the feedback loop sit alongside.

6. Data flow. OULAD's seven tables become snapshot features at each checkpoint, using only data dated before that day. The key rule: we only score students who are still enrolled. If you also score students who already left, ROC-AUC looks like 0.749 instead of 0.726, because 75% of withdrawals had already happened. That inflated number is useless to an advisor.

7. Technology stack. Claude in three tiers by measured need: Haiku parses search requests, Sonnet writes the agent text, Opus judges. e5-small-v2 embeddings won a five-model comparison. LanceDB is an embedded vector database with filters pushed into the search. LangGraph runs the agent graph. XGBoost with TreeSHAP for the models. FastAPI, and React for the console, served by the same process. DeepEval and RAGAS for evaluation.

8. Agent workflow. The performance agent summarises strengths and concerns; the risk agent explains the calibrated risk with precedents; a router decides intervene or monitor; the intervention agent chooses 2 to 4 actions from an eligible catalogue and drafts the message; the critic checks IDs, numbers and wording in code; one revision, then a deterministic fallback. Agents talk through a typed shared state with numbered evidence, which is what makes claims checkable.

9. Hybrid RAG. Three channels: behavioural vectors weighted by what predicts outcomes, BM25 over descriptor tokens, and e5 dense embeddings of narratives. Fused with weighted reciprocal rank fusion, filtered in LanceDB, earlier presentations only. The retrieved students' real outcomes predict the query student's with AUC 0.835 vs 0.512 for random. Plain-English search reaches precision@10 0.862.

10. Prediction models. Withdrawal risk at week 15: ROC-AUC 0.726 on a later, untouched presentation, against 0.703 for logistic regression, and calibrated: expected calibration error 0.009, so a 20% risk means about 20%. The High tier catches 50% of withdrawals while flagging 24% of students. Accuracy rises through the course, from 0.686 at week 5 to 0.773 at week 22.

11. Demo. Follow section 3.

12. Evaluation results. Five layers. Code checks on every output: 100% of plans pass. LLM-as-judge (Opus, a different model): 96% pass; in an order-swapped A/B the AI plan is preferred over the computed plan 89% of the time. DeepEval faithfulness 100% pass. RAGAS: precedent syntheses are 0.964 faithful to the retrieved cases (hallucination rate 3.6%). 139 automated tests run with no network and no LLM.

13. Responsible AI. Sensitive attributes are never model inputs and never sent to the LLM; adding them would change ROC-AUC by only 0.002. Every LLM output is checked for protected, punitive and clinical wording. The fairness audit tests each gap against chance: the disability gap is real and reported, with a threshold review for the equity office, not hidden or auto-corrected.

14. Challenges and learnings. The biggest lesson: honest evaluation first. An earlier version reported 0.90 AUC by scoring students who had already left. Second: numbers do not belong in text embeddings, so we band them into descriptors and keep numeric similarity in its own channel. Third: let the LLM narrate and select, and let code decide facts. Fourth: a feature set that improved validation was rejected because it did not hold on the test presentation.

15. Thank you. "Happy to take questions, or show any part of it live."

3. Live demo script (2 minutes)

Before the session: `uvicorn src.main:app --reload`, open `http://127.0.0.1:8000/` and the console at `/today`, choose **BBB 2014J (held out)** and **week 15**. Close other tabs. If the network or the API key fails, set `LLM_ENABLED=false` in `.env` and restart: every feature still works with computed text (labelled "Computed").

1. **Landing page** (5 s): "Live numbers from the API; the dome is real weekly activity of students who passed and withdrew."
2. **This week** (20 s): the course clock across the top; click week 5 then back to week 15 and point out that every number changes; show the High-tier count, expected withdrawals and "Who to reach first".
3. **Student case file** (40 s): open the first student. Why flagged: drivers. Across the course: risk at each week. Click **Draft a support plan**: the agent steps appear one by one; show actions, the outreach message labelled "Drafted by AI", rewrite it as an SMS, and the critic's checks. Scroll to **Similar students**: what happened to them next.
4. **Case finder** (25 s): click the example "students who stopped turning in their coursework": the request is understood as filters and behaviours; results with outcomes and trajectories.
5. **Copilot** (20 s): ask "Who should I contact first in BBB 2014J?": tool calls appear, the answer streams, IDs link to case files, "Every number checked".
6. **Model quality** (10 s): AI features tab: judge, DeepEval and RAGAS results, if time allows.

4. Key numbers

Number	Value	Meaning
Enrolments / in scope at week 15	32,593 / 25,204	Students still enrolled are the only ones scored
Withdrawal ROC-AUC (95% CI)	0.726 (0.708 to 0.740)	Ranking quality on the untouched test presentation
Logistic baseline ROC-AUC	0.703	What a simple model gets
PR-AUC (base rate)	0.263 (0.113)	2.3x better than chance at finding withdrawals

Number	Value	Meaning
Calibration error (ECE)	0.009	Predicted risks match observed rates
High tier	24% of students, 50% of withdrawals	Where to look first
Ranking precision at 10%	98% (random 34%)	Share of contacted students who later failed or withdrew
Outcome model macro-F1	0.485 (logistic 0.460)	Four classes, hard problem; used for fail-or-withdraw
Retrieval neighbour-vote AUC	0.835 (random 0.512)	Retrieved students' outcomes predict the query student's
Typed search precision@10	0.862	Plain-English case search
Judge pass rate / A/B win	96% / 89%	Quality of AI plans
DeepEval faithfulness / relevancy	1.00 / 1.00	Grounded and on-topic
RAGAS faithfulness (synthesis)	0.964	Claims supported by retrieved cases
Adversarial prompts handled safely	88% of 8	The one miss is a refusal that repeated a word
Automated tests	139	No network, no LLM

5. Glossary

Term	Plain meaning
Checkpoint	A point in the course (share of its length) at which we take a snapshot and predict; here weeks 5, 9, 15, 22
In scope	Still enrolled at the checkpoint; the only students we score
ROC-AUC	Chance that a random student who withdraws is ranked above one who does not (0.5 = random, 1 = perfect)
PR-AUC	Precision averaged over recall; compare it with the base rate, not with 1
ECE	Expected calibration error: average gap between predicted and observed rates
Lift	Precision at a capacity divided by the base rate
TreeSHAP	Exact per-feature contributions to one prediction of a tree model
BM25	Classic keyword ranking function used by search engines
RRF	Reciprocal rank fusion: combine ranked lists by summing 1/(k + rank); no score calibration needed
Neighbour-vote AUC	Use the retrieved students' real outcomes as a prediction for the query student; its AUC measures retrieval quality without circular labels
nDCG@10	Ranking quality of the top 10 against an ideal order
Faithfulness (RAGAS / DeepEval)	Share of claims in the answer that the context supports

Term	Plain meaning
LLM-as-judge	A separate model scoring outputs against a rubric
Order-swapped A/B	Asking the judge to compare A vs B twice, in both orders, to cancel position bias
Cohen's kappa	Agreement between two raters beyond chance

6. Panel questions and answers

Data and leakage

Q: Why is your AUC lower than numbers reported elsewhere on OULAD? Many reports score every enrolment, including students who had already withdrawn, and use random splits. Scored that way, our model would show 0.749. We only score students still enrolled at the checkpoint and test on a later presentation, which is the situation an advisor is in: 0.726 is the honest number.

Q: How do you prevent leakage? Features use only rows dated on or before the checkpoint day; students who left before it are out of scope; `date_unregistration` defines scope only and is never a feature; `final_result` is only a label; peer percentiles are computed among in-scope peers at that time; the test presentation is never used for any choice; retrieval only looks at earlier presentations.

Q: Why these checkpoints? Advisors need warnings early, but early predictions are weaker. Four checkpoints (weeks 5, 9, 15, 22) each with their own model let the team choose; the console's course clock switches between them.

Q: Why a temporal split instead of cross-validation? Students in the same presentation share a course and a calendar; a random split leaks that. Training on 2013, tuning on 2014B and testing on 2014J mimics deploying on next year's cohort.

Models

Q: Why XGBoost? Best ranking on these tabular features, exact TreeSHAP explanations, fast on a CPU. It beats a logistic regression baseline (0.726 vs 0.703), which we keep as a check.

Q: Why calibrate? Advisors read percentages. After calibration the expected calibration error is 0.009, so "25% risk" means roughly one in four. Uncalibrated scores would only be good for ranking.

Q: How are tiers chosen? On the validation presentation: Medium catches 80% of withdrawals; High maximises F1 above that. On test the tiers separate cleanly: observed withdrawal rate 23.6% High, 11.7% Medium, 4.9% Low.

Q: Why is outcome macro-F1 only 0.485? Four classes with Distinction and Fail being rare and close to Pass at mid-course. It beats both baselines; what we use from it, P(fail or withdraw), has ROC-AUC 0.852.

Q: Could you get higher scores? We tried richer features and a larger hyperparameter search. Validation improved but calibration and macro-F1 got worse on the untouched test presentation, so we rejected them rather than tune on the test set. The current numbers are close to the ceiling for this honest protocol.

Q: How does prioritisation work? A weighted blend of withdrawal risk, fail-or-withdraw probability and actionable gaps (scores, activity, missed work, recent decline), weights chosen on validation by an exhaustive grid with a one-standard-error rule that prefers a blend when it is statistically as good as the best single signal.

RAG and retrieval

Q: What is your chunking strategy? One document per enrolment, no splitting: a case is a short record and splitting it would separate signals from the outcome. Numbers are banded into descriptor tokens; a short narrative is embedded.

Q: Why hybrid and not just embeddings? Student profiles are numeric; text embeddings do not understand that 45 and 46 are close. The behavioural channel handles numeric similarity, BM25 matches exact descriptors, dense handles free-text phrasing. RRF combines them without calibrating scores.

Q: How do you know retrieval works? We use the retrieved students' real later outcomes to predict the query student's: AUC 0.835 vs 0.512 random, on 600 test students. This is not circular because relevance comes from outcomes, not from the same features. Typed search has 40 labelled queries.

Q: Why LanceDB? Embedded (no server to run), stores the metadata next to the vectors so filters run as a pre-filter (they never remove true neighbours), exact search at this size, identical results to a brute-force scan in our parity test.

Q: Why e5-small-v2? We compared five embedding models on the same narratives: e5-small-v2 had the best precedent quality and near-best typed search at the same latency as MiniLM; the larger mpnet was slower and not better.

Q: Why 6 cases in the context and 50 for the outcome mix? A sensitivity study: more neighbours give steadier outcome estimates (so 50 for statistics), but a handful of concrete stories keeps prompts readable and cheap (so 6).

Q: Can retrieval see the future? No. Only presentations that finished before the query's presentation are searchable, and the student's own rows are excluded.

Agents and the LLM

Q: Why multiple agents? Each step has a different job and a different check: summarising performance, explaining risk, choosing eligible actions. Splitting them makes each output small, schema-bound and verifiable, and lets the critic target the step that failed.

Q: How do agents communicate? Through a typed shared state in LangGraph: numbered observations and eligible actions go in, each agent writes structured output that cites those IDs, the critic writes violations. There is no free-form chat.

Q: What stops hallucination? The LLM only sees computed evidence and must cite observation IDs; code rejects unknown IDs, ineligible actions and any number not present in the evidence; one revision, then the computed plan. RAGAS measures what is left: 3.6% for precedent syntheses.

Q: What if the LLM is down or too expensive? `LLM_ENABLED=false`: every feature still works with computed text, labelled as such. Responses are cached, prompt prefixes are cached, a budget cap stops evaluation runs, and the ledger shows cost per feature.

Q: Why three Claude models? Measured, not assumed: for parsing search requests the small model is as accurate as the large ones at a fraction of the cost; for precedent synthesis the main model had the highest judge pass rate; the judge must be a different, stronger model so it does not grade itself.

Q: Why LangGraph and not LangChain or LlamaIndex agents? We needed explicit control flow (router, critic, revision, fallback) around our own validated components. LangGraph gives that graph without replacing the parts we measured; a parity test shows it matches the plain-Python orchestrator exactly.

Evaluation

Q: Why both DeepEval and RAGAS? DeepEval scores the agent analyses and plans (faithfulness, relevancy and custom criteria such as "supportive and non-punitive"); RAGAS scores the retrieval-augmented outputs against their retrieved context. Different questions, different tools.

Q: Can you trust an LLM judge? We reduce its biases: a different model from the generators, an anchored rubric, the verdict computed from its scores in code, order-swapped A/B to cancel position bias, and a human-review tab that measures agreement (Cohen's kappa) once advisors rate cases.

Q: Why is copilot RAGAS faithfulness lower (0.834)? Copilot answers are scored whole, so sentences explaining what a metric means count as unsupported. The numbers in copilot answers are separately checked in code against the tool results, and the golden questions are 100% correct.

Q: What do the tests cover? 139 tests: every API route, validation and auth, the pipeline's leakage controls, models, retrieval guards, vector-store parity, the agent workflow with fake LLM outputs (valid, hallucinating, unsafe), guardrails, the Claude client with a mocked SDK, fairness, feedback and jobs. No network, no LLM, isolated paths.

Guardrails, fairness and ethics

Q: Do you use gender, age or disability? Never as model inputs and never in prompts. They are used only to audit fairness and as optional filters in case search.

Q: Is the model fair? We test each group gap against chance. Region and deprivation gaps are within chance at these group sizes. The disability gap (0.23) is real: disabled students withdraw more and have more of their withdrawals caught. We show what a per-group cut-off would change, for the equity office to decide; we do not apply it automatically because that would use a sensitive attribute in decisions.

Q: Could this be used to penalise students? It is designed against that: the action catalogue is supportive only, outreach never mentions risk scores or withdrawal, punitive and clinical terms are blocked, and an advisor reviews every message. Intended use excludes grading, admissions and sanctions.

Q: What input validation is there? API key, admin key for changes, rate limiting, request-size limit, strict schemas (module and presentation vocabularies, ID and value ranges), injection screening of free text and generic error messages.

Architecture and engineering

Q: Why React and not Streamlit? Streamlit would satisfy the brief; React lets the console do more: per-student URLs, virtualised tables, streamed copilot answers with tool calls, keyboard and screen-reader support. It is served by FastAPI, so the demo still runs with one command and needs no Node.js.

Q: How would this scale? Models load once per checkpoint; requests are fast except AI drafting, which runs as background jobs. Several workers on one host share jobs and rate limits through SQLite; beyond that, a job queue and gateway limits (documented in `docs/DEPLOYMENT.md`).

Q: How does the feedback loop avoid making things worse? Proposals need minimum samples, and nothing changes until an admin applies them; applied changes are versioned and the offline evaluation is re-run.

Q: What would you do next? Collect real advisor feedback and human labels, revalidate on newer cohorts, and add an approximate-nearest-neighbour index if the case base grows.